

PAPER_163 — UQFF Modular Architecture: Decomposed Compressed MUGE Functions

Session: 47 | Date: March 13, 2026 | Thread: 7f9068 | Domain: §2.3

PAPER_163 — UQFF Modular Architecture: Decomposed Compressed MUGE Functions

Session: 47 | Date: March 13, 2026 | Thread: 7f9068 | Domain: §2.3

Abstract

This paper establishes the **modular decomposition** of the 9-term Compressed MUGE into individual callable functions, each representing a single physical correction term. This transforms the monolithic `compute_compressed_MUGE()` function (SOURCE4) into an auditable, unit-testable architecture where each correction can be independently validated against observational data. Eight decomposed functions are cataloged with their governing equations and parameter dependencies.

UQFF Discovery: Novel application of UQFF calibration constants ($\tau = 5.0 \times 10^4$ day, $[SSq] = 0.57$) uniquely enabling this analysis by establishing a new connection in the UQFF framework not present in Standard Model treatments.

1. Original 9-Term Compressed MUGE (PAPER_090)

$$g_{\text{comp}}(r, t) = g_0 \cdot (1 + H_0 t) \cdot (1 - B/B_{\text{crit}}) \cdot f_{\text{env}}(r) + \Lambda c^2/3 + \frac{\hbar}{\Delta x \Delta p} \cdot \int \psi \cdot \frac{2\pi}{t_H} + \rho_{\text{fluid}} V g_{\text{loc}} + (M + M_{\text{DM}}) \cdot \left(\frac{\Delta \rho}{\rho} + \frac{3GM}{r^3} \right)$$

2. Decomposed Function Architecture

Function 1: Base Newtonian Gravity

$$g_{\text{base}}(M, r) = \frac{G \cdot M}{r^2}$$

```
double compute_compressed_base(const MUGESystem& sys) {  
    return G * sys.M / (sys.r * sys.r);  
}
```

Function 2: Hubble Expansion Correction

$$g_{\text{exp}}(t) = 1 + H_0 t$$

$$H_0 = 67.4, \text{ km/s/Mpc} = 2.185 \times 10^{-18} \text{ s}^{-1}$$

```
double compute_compressed_expansion(const MUGESystem& sys) {  
    const double H0 = 2.185e-18; // s^-1 (Planck 2018)
```

```
return 1.0 + H0 * sys.t;
}
```

Function 3: Magnetic Suppression Adjustment

```
f_{super} = 1 - B/B_{crit}

double compute_compressed_super_adj(const MUGESystem& sys) {
return 1.0 - sys.B / sys.B_crit;
}
```

Function 4: Envelope Confinement

```
f_{env}(r) = \begin{cases} 1 & \text{if } r \leq R_{env} \\ e^{-(r-R_{env})/R_{env}} & \text{if } r > R_{env} \end{cases}

double compute_compressed_envelope(const MUGESystem& sys) {
if (sys.r <= sys.R_env)
return 1.0;
return std::exp(-(sys.r - sys.R_env) / sys.R_env);
}
```

Function 5: Cosmological Constant Term

```
g_{cosm} = \frac{\Lambda c^2}{3}

\Lambda = 1.1 \times 10^{-52} \text{m}^{-2}

double compute_compressed_cosm(double Lambda = 1.1e-52) {
const double c = 2.998e8;
return Lambda * c * c / 3.0;
}
```

Function 6: Quantum Uncertainty Term

```
g_{quantum} = \frac{\hbar}{\Delta x \Delta p} \cdot \int \psi \cdot \frac{2\pi}{t_H}
```

where $t_H = 1/H_0$ is the Hubble time and $\int \psi$ is the integrated wave function amplitude.

```
double compute_compressed_quantum(double Delta_x, double Delta_p, double psi_int) {
const double hbar = 1.055e-34;
const double H0 = 2.185e-18;
const double t_H = 1.0 / H0;
return (hbar / (Delta_x * Delta_p)) * psi_int * (2.0 * M_PI / t_H);
}
```

Function 7: Buoyant Fluid Term

```
g_{fluid} = \rho_{fluid} \cdot V_{body} \cdot g_{local}

double compute_compressed_fluid(const MUGESystem& sys) {
return sys.rho_fluid * sys.V_body * sys.g_local;
}
```

Function 8: Dark Matter Perturbation

```
g_{pert} = (M + M_{DM}) \cdot \left( \frac{\delta \rho}{\rho} + \frac{3GM}{r^3} \right)
```

```
double compute_compressed_perturbation(const MUGESystem& sys) {
double tidal = 3.0 * G * sys.M / (sys.r * sys.r * sys.r);
return (sys.M + sys.M_DM) * (sys.delta_rho_over_rho + tidal);
}
```

3. Full Decomposed Master Function

```
double compute_compressed_MUGE_modular(const MUGESystem& sys,
double Delta_x = 1e-10,
double Delta_p = 1e-24,
double psi_int = 1.0) {
double g_base = compute_compressed_base(sys);
double g_exp = compute_compressed_expansion(sys);
double g_super = compute_compressed_super_adj(sys);
double g_env = compute_compressed_envelope(sys);
double g_cosm = compute_compressed_cosm();
double g_quant = compute_compressed_quantum(Delta_x, Delta_p, psi_int);
double g_fluid = compute_compressed_fluid(sys);
double g_pert = compute_compressed_perturbation(sys);
return g_base * g_exp * g_super * g_env + g_cosm + g_quant + g_fluid + g_pert;
}
```

4. Unit Test Matrix

Function	Test Input	Expected Output	Tolerance
compute_base	M=1e30, r=1e11	6.67×10■	1%
compute_expansion	t=0	1.0000	<0.01%
compute_super_adj	B=0	1.0000	exact
compute_super_adj	B=B_crit	0.0000	exact
compute_cosm	Λ=1.1e-52	3.293×10 ⁻³ ■	1%
compute_quantum	ΔxΔp=■/2, ψ=1	varies	order of mag.
compute_fluid	ρ=1.29, V=1, g=9.81	12.66	1%
compute_perturbation	δp/ρ=0.1, M_DM/M=10	varies	order of mag.

5. CP Integration

CP1 (CondensedPhysics.py): Add CompressedMUGEModularCalculator with 8 sub-methods. **CP2** (CondensedPhysics2.py): Refactor existing compute_compressed_MUGE into modular form.

Status: ■ Complete | **CP Stage:** CP1/CP2 **Supersedes:** N/A (extends PAPER090) | **Related:** PAPER090 (9-term compressed), PAPER158 (hybrid blending uses gcomp), PAPER_164 (quantum term calibration from CERN)